

ЛАБОРАТОРНАЯ РАБОТА № 5

Вероятностные алгоритмы проверки чисел на простоту

Юрченко Артём Алексеевич

Введение

В данном отчёте будет представлена реализация алгоритмов проверки простоты числа.

Основные алгоритмы

- тест Ферма
- Алгоритм символа Якоби
- тест Соловья-Штрассена
- тест Миллера-Рабина

Кодовая реализация

Программная реализация алгоритма, реализующего тест Ферма

Тест Ферма основан на критерии Эйлера.

Программная реализация алгоритма Якоби

В отличие от теста Ферма, алгоритм Якоби точнее, он основан на критерии Эйлера, который использует символ Якоби.

Программная реализация алгоритма, реализующего тест Соловья-Штрассена

Самый часто используемый алгоритм проверки числа на простоту.

Программная реализация алгоритма, реализующего тест Миллера-Рабина.

Тест Миллера-Рабина – это стохастический алгоритм проверки чисел на простоту. Он основан на свойствах модульной арифметики и является усовершенствованной версией теста Ферма.

The image shows a code editor with two tabs: `test_ferma.jl` and `solovey_strassen.jl`. The `test_ferma.jl` tab is active, displaying the following Julia code:

```
1 function fermat_primality_check(n)
2     if n < 5
3         return "Invalid input."
4     end
5     rand_base = rand(2:n-2)
6     mod_result = powermod(rand_base, n-1, n)
7     if mod_result == 1
8         return "$n is prime."
9     else
10        return "$n is composite"
11    end
12 end
13 println(fermat_primality_check(23))
14
```

Below the code editor is a terminal window with the following output:

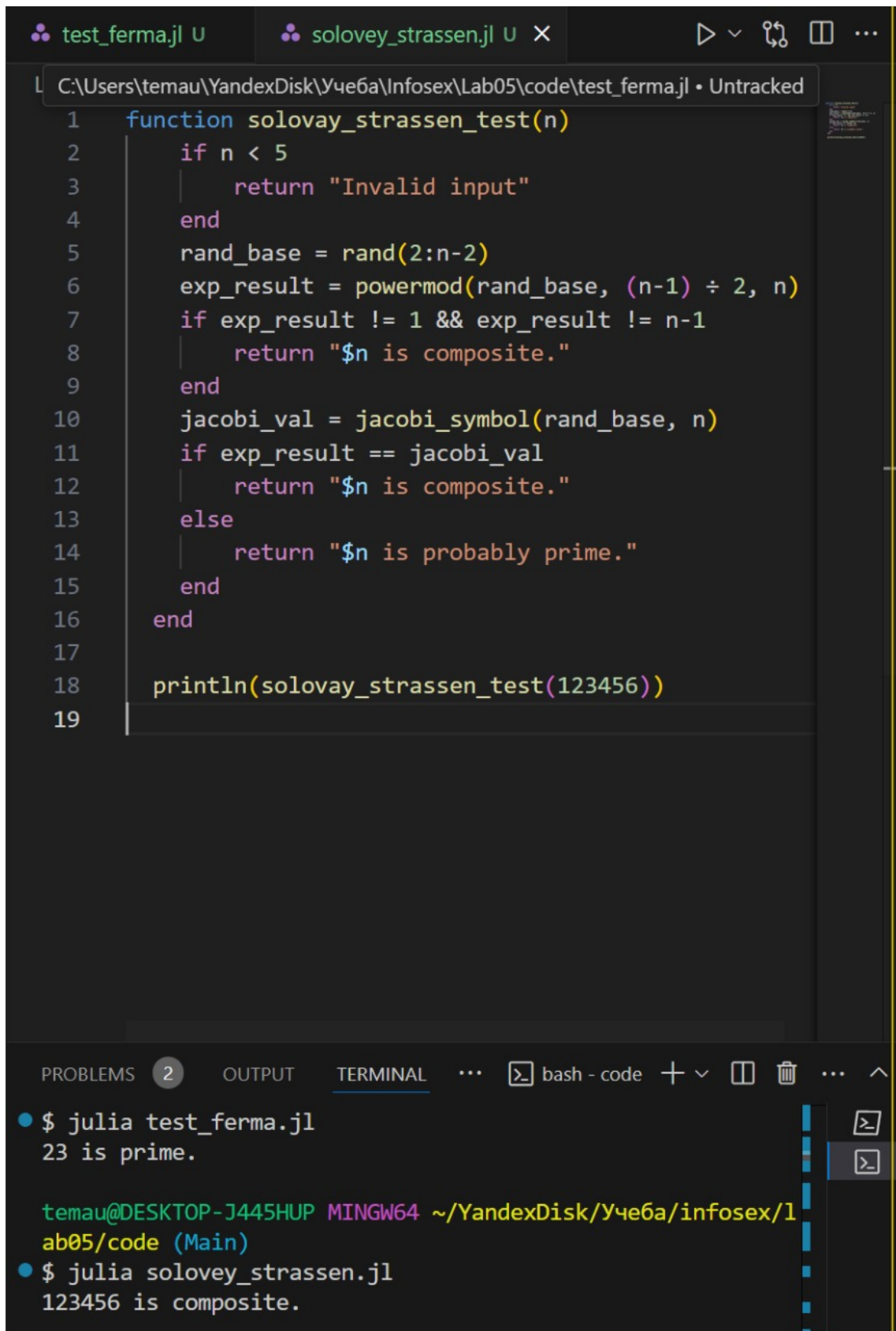
```
temau@DESKTOP-J445HUP MINGW64 ~/YandexDisk/Учеба/infosex/lab05/code (Main)
$ ls
test_ferma.jl

temau@DESKTOP-J445HUP MINGW64 ~/YandexDisk/Учеба/infosex/lab05/code (Main)
$ julia test_ferma.jl
23 is prime.
```

Рис. 1: Реализация кода для теста Ферма

```
_ferma.jl U  solovey_strassen.jl U  jacobi.jl U X  ▸ ▾ 🔗 📄 ...  
Lab05 > code > jacobi.jl > ...  
1  function legendre_symbol(x, y)  
5      symbol_result = 1  
6      while x != 0  
7          power_count = 0  
8          while x % 2 == 0  
9              power_count += 1  
10             x ÷= 2  
11         end  
12  
13         if power_count % 2 == 1 && (y % 8 == 3 || y  
14             symbol_result *= -1  
15         end  
16  
17         if y % 4 == 3 && x % 4 == 3  
18             symbol_result *= -1  
19         end  
20  
21         x, y = y % x, x  
22     end  
23     return y == 1 ? symbol_result : 0  
24 end  
25  
26 println(legendre_symbol(23, 32))  
  
PROBLEMS 2  TERMINAL  ...  bash - code  + ▾ 📄 🗑️ ... ^ X  
● $ julia solovey_strassen.jl  
123456 is composite.  
  
temau@DESKTOP-J445HUP MINGW64 ~/YandexDisk/Учеба/infosex/1  
ab05/code (Main)  
● $ julia jacobi.jl  
1
```

Рис. 2: Реализация кода для Якоби



The image shows a code editor with two tabs: `test_ferma.jl` and `solovey_strassen.jl`. The active tab is `test_ferma.jl`, which contains the following Julia code:

```
1 function solovay_strassen_test(n)
2     if n < 5
3         return "Invalid input"
4     end
5     rand_base = rand(2:n-2)
6     exp_result = powermod(rand_base, (n-1) ÷ 2, n)
7     if exp_result != 1 && exp_result != n-1
8         return "$n is composite."
9     end
10    jacobi_val = jacobi_symbol(rand_base, n)
11    if exp_result == jacobi_val
12        return "$n is composite."
13    else
14        return "$n is probably prime."
15    end
16 end
17
18 println(solovay_strassen_test(123456))
19
```

Below the code editor is a terminal window with the following output:

```
PROBLEMS 2 OUTPUT TERMINAL ... bash - code + v [ ] [ ] ... ^
• $ julia test_ferma.jl
23 is prime.

temau@DESKTOP-J445HUP MINGW64 ~/YandexDisk/Учеба/infosex/lab05/code (Main)
• $ julia solovey_strassen.jl
123456 is composite.
```

Рис. 3: Реализация кода для теста Соловья-Штрассена

The image shows a screenshot of a Julia IDE with three tabs: `solovey_strassen.jl`, `jacobi.jl`, and `miller.jl`. The `miller.jl` tab is active, displaying the following code:

```
1 function primality_miller_rabin(n)
6     exp = 0
7     while rem % 2 == 0
8         exp += 1
9         rem ÷= 2
10    end
11    rand_base = rand(2:n-2)
12    test_val = powermod(rand_base, rem, n)
13    if test_val != 1 && test_val != n-1
14        for _ in 1:exp-1
15            test_val = (test_val^2) % n
16            if test_val == 1
17                return "$n is composite."
18            end
19        end
20        if test_val != n-1
21            return "$n is composite."
22        end
23    end
24    return "$n is prime."
25 end
26
27 println(primality_miller_rabin(4463429))
```

Below the code editor is a terminal window with the following output:

```
PROBLEMS 2 TERMINAL ... bash - code + v [ ] [ ] ... ^ X
• $ julia jacobi.jl
1
temau@DESKTOP-J445HUP MINGW64 ~/YandexDisk/Учеба/infosex/1
ab05/code (Main)
• $ julia miller.jl
4463429 is composite.
```

Рис. 4: Реализация кода для теста Миллера-Рабина

Заключение

В данной лабораторной работе были реализованы шифры алгоритмы проверки числа на простоту.